

































































































































































































































































































































































































































Yes! This is the right question to ask now. 😊 Let's make the whole thing concrete.

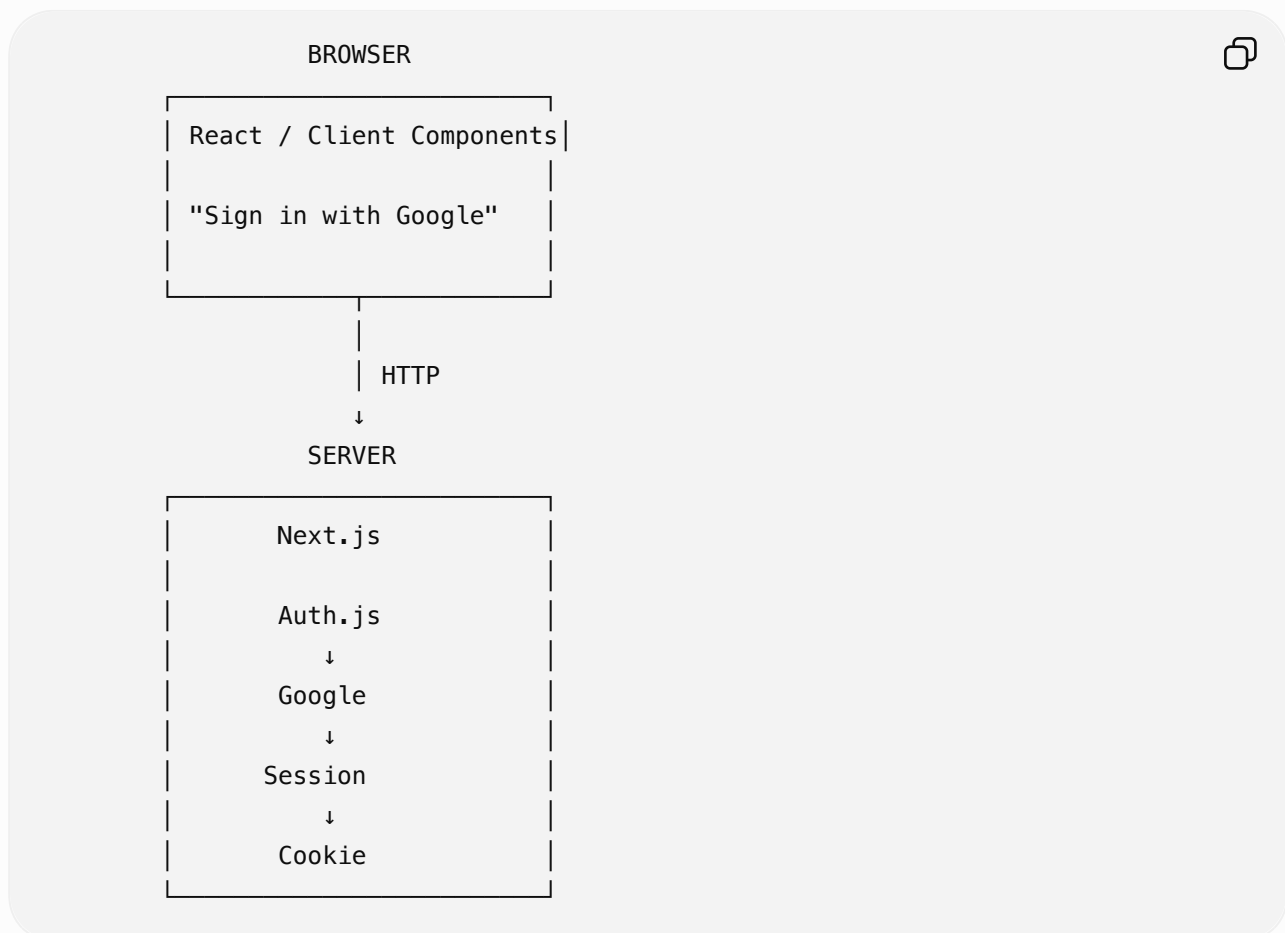
I checked the **current Auth.js documentation** because the package/setup has evolved. For Next.js, the current Auth.js integration uses `next-auth`, with the newer configuration style centered around an `auth.ts` file and exports such as `auth`, `handlers`, `signIn`, and `signOut`.  Auth.js

## 1. First: where does Auth.js actually exist?

Auth.js runs on the server.

Not as your authentication engine in the browser.

Think:



The browser **does participate in the OAuth flow** because the browser gets redirected to Google and back.

But **Auth.js itself is server-side application code**.

That's an important distinction.

---

## 2. OAuth full form

**OAuth = Open Authorization.**

OAuth is a protocol/framework that allows an application to obtain **authorized access** to resources without the user giving the application their provider password.

For login, what we're commonly doing is **OAuth 2.0 / OpenID Connect (OIDC)**.

This distinction matters:

- **OAuth 2.0** → primarily an authorization framework.
- **OpenID Connect (OIDC)** → an identity layer built on OAuth 2.0, commonly used when you want to know **who the user is**.

Google login commonly involves OIDC.

So don't memorize:

"OAuth = authentication."

A better interview answer is:

"**OAuth 2.0 is an authorization framework, while OpenID Connect adds an identity layer on top of OAuth 2.0 and is commonly used for login.**"

---

## 3. Now let's build the entire thing

Let's imagine:

```
My Next.js App
+
Auth.js
+
Google
```



Our goal:



```
Gopi
↓
Click "Continue with Google"
↓
Google authenticates Gopi
↓
Google redirects back
↓
Auth.js processes callback
↓
Auth.js creates application session
↓
Session cookie
↓
Gopi can access /dashboard
```

Let's actually see the code.

## 4. Install Auth.js

For a Next.js application, the package is:

```
npm install next-auth
```

The current Auth.js Next.js setup uses `next-auth` .  Auth.js

## 5. Project structure

We'll have something like:



```
my-app/
|
├── app/
|   ├── api/
|   │   ├── auth/
|   │   │   ├── [...nextauth]/
|   │   │   └── route.ts
|   ├── login/
|   │   └── page.tsx
|   └── dashboard/
|       └── page.tsx
|
```

```
|— auth.ts  
|— .env.local
```

The key file is:

auth.ts



## 6. auth.ts — configuring Auth.js

```
// auth.ts  
  
import NextAuth from "next-auth";  
import Google from "next-auth/providers/google";  
  
export const {  
  handlers,  
  auth,  
  signIn,  
  signOut,  
} = NextAuth({  
  providers: [  
    Google({  
      clientId: process.env.AUTH_GOOGLE_ID,  
      clientSecret: process.env.AUTH_GOOGLE_SECRET,  
    }),  
  ],  
});
```

This is essentially telling Auth.js:

"Google is an authentication provider for my application."

The current Auth.js documentation uses this `NextAuth({...})` configuration pattern.



## 7. Where do these Google credentials come from?

You create an OAuth application in Google's developer console.

You receive something conceptually like:

```
GOOGLE_CLIENT_ID
GOOGLE_CLIENT_SECRET
```



Then:

```
AUTH_GOOGLE_ID=xxxxxxx
AUTH_GOOGLE_SECRET=xxxxxxx
AUTH_SECRET=xxxxxxx
```

## VERY important

The client secret stays on the server.

Don't do:

```
"use client";

const secret = "my-google-secret";
```

✗ Never expose that.

---

## 8. Now the Route Handler

This is where people often go:

"Wait, why do we need this route.ts?"

Because Auth.js needs HTTP endpoints through which the browser can initiate authentication and receive OAuth callbacks.

Current Auth.js's Next.js example uses:

```
// app/api/auth/[...nextauth]/route.ts

import { handlers } from "@auth";

export const { GET, POST } = handlers;
```

That's the current documented pattern.  Auth.js

So now your application has authentication endpoints under:

```
/api/auth/...
```



Auth.js handles the details.

## 9. Login button

We can initiate Google sign-in using the `signIn` function exported by Auth.js.

For example, conceptually:

```
"use client";

import { signIn } from "next-auth/react";

export default function LoginButton() {
  return (
    <button onClick={() => signIn("google")}>
      Continue with Google
    </button>
  );
}
```

Now let's trace exactly what happens.

## 10. User clicks the button

Browser



```
[ Continue with Google ]
      ↓
signIn("google")
```

This initiates the Auth.js sign-in flow.

The browser is then directed toward the Auth.js authentication endpoint.

Conceptually:

Browser



↓

/api/auth/signin/google

↓

Auth.js

## 11. Auth.js knows Google is configured

Remember this:

```
providers: [  
  Google({  
    clientId: "...",  
    clientSecret: "...",  
  })  
]
```

So Auth.js knows:

Provider = Google



and knows Google's OAuth/OIDC configuration.

It constructs the appropriate authorization request.

Then:

Browser

↓

Google



## 12. Browser is now on Google

The browser might be redirected to something conceptually like:

accounts.google.com/...



The exact URL is much more complicated, but don't worry about memorizing it.

The important thing is:

YOUR APP

↓





Google

## 13. User logs into Google

Google displays:



The password goes to:

Google

**NOT your Next.js server.**

That's one of the major benefits of using an external identity provider.

## 14. Google authenticates the user

Google says:

"Yes, this user successfully authenticated."

But Google doesn't simply send:

`user = Gopi`

and expect your application to blindly trust it.

The OAuth/OIDC flow uses things such as an **authorization code** and token exchange/validation.

Conceptually:

```
Google
  ↓
authorization code
  ↓
Browser
  ↓
Your callback URL
```



For example:

```
GET /api/auth/callback/google?code=ABC123
```



---

## 15. Here's where your Route Handler enters

Remember:

```
// app/api/auth/[...nextauth]/route.ts

import { handlers } from "@auth";

export const { GET, POST } = handlers;
```

The request:

```
GET /api/auth/callback/google?code=ABC123
```



reaches:

```
Next.js Route Handler
  ↓
Auth.js
```



Auth.js recognizes:

```
callback
provider = google
```



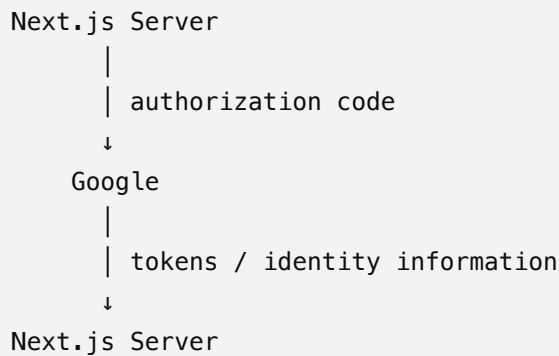
and processes it.

---

## 16. Auth.js communicates with Google

Now the **server** can communicate with Google's token endpoint.

Conceptually:



The important thing is that the sensitive provider credentials and token exchange happen server-side.

## 17. Auth.js now knows who authenticated

Google provides identity information through the OAuth/OIDC flow.

Conceptually:

```
graph TD; A[Google] --> B["{ id: '123', email: 'gopi@example.com', name: 'Gopi' }"];
```

The diagram shows Google returning identity information in the form of a JSON object:

```
{  id: "123",  email: "gopi@example.com",  name: "Gopi"}
```

Auth.js can then associate that identity with a user in your application.

## 18. Where is the user stored?

This depends on your Auth.js configuration.

This is another thing I want you to understand:

**Authentication library  $\neq$  automatically one specific database architecture.**

You can configure Auth.js with an adapter to persist users/accounts/sessions in a database.

For example, conceptually:

```
Google account
  ↓
Auth.js
  ↓
Database
```



Your DB might contain tables/entities along the lines of:

```
User
-----
id
name
email

Account
-----
userId
provider
providerAccountId
```



So your application can remember:

```
Google account ABC
  ↓
Application user 123
```



---

## 19. What about the session?

After successful authentication, Auth.js establishes a session according to your configured session strategy.

Conceptually:

```
Authenticated user
  ↓
Application session
  ↓
Session cookie
  ↓
Browser
```



There are different session strategies, including **JWT-based sessions** and **database sessions**, depending on configuration.

This is important because earlier we learned:

"Session ID → user ID stored in DB."

That is **one possible session architecture**, not the only architecture Auth.js can use.

## 20. Database session version

Let's first imagine the database-session model because it matches what we've already learned.

Database:

Session

| sessionToken | userId | expires |
|--------------|--------|---------|
| ABC123       | 42     | ...     |



Browser:

Cookie:

sessionToken=ABC123



Then:

Browser

GET /dashboard  
Cookie: sessionToken=ABC123

↓

Next.js

↓

Auth.js

↓

Session lookup

↓

userId = 42

↓

User

↓

Gopi



That's very similar to the session architecture we built manually.

## 21. JWT session version

Another possibility is:

```
Browser
  |
  | session cookie
  ↓
Next.js
  ↓
Auth.js
  ↓
JWT/session information
  ↓
Current user
```



The session information can be represented in a signed/encrypted token rather than requiring a database lookup for every session lookup, depending on configuration.

So:

```
Database session
```



and

```
JWT session
```



are different session strategies.

Don't assume:

```
Auth.js = database session.
```

It doesn't.

---

## 22. Now /dashboard

Suppose we have:

```
// app/dashboard/page.tsx

import { auth } from "@auth";

export default async function Dashboard() {
```

```
const session = await auth();

if (!session?.user) {
  return <div>Please log in</div>;
}

return (
  <div>
    <h1>Welcome {session.user.name}</h1>
    <p>{session.user.email}</p>
  </div>
);
}
```

Notice this:

```
const session = await auth();
```

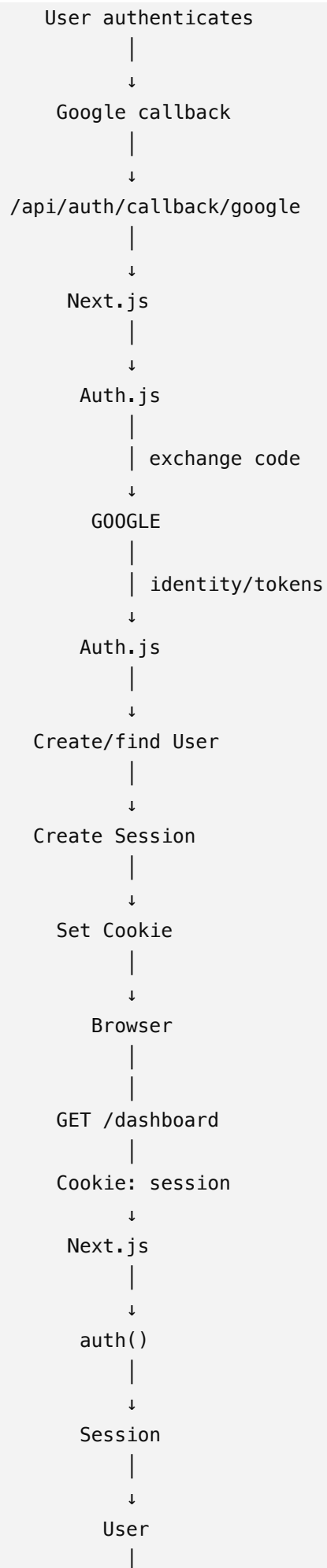
That's running on the **server** because this is a Server Component.

The current Auth.js documentation specifically demonstrates using `auth()` from a Server Component to obtain session data. [NextAuth.js Exam...](#)

## 23. Full flow — put EVERYTHING together

Now let's connect the dots.







↓  
Dashboard

🔥 That's the entire picture.

---

## 24. And where is Auth.js?

Now you can answer your original question precisely:

**Auth.js itself:**

SERVER



**Google login page:**

BROWSER



**Browser:**

redirects between your app and Google  
stores/sends your application session cookie



**Google:**

IDENTITY PROVIDER



**Next.js:**

APPLICATION SERVER



**Auth.js:**

AUTHENTICATION FRAMEWORK RUNNING WITH YOUR NEXT.JS SERVER



---

## 25. One subtle thing: "provider" doesn't necessarily mean Google

This is worth locking in.

When we say:

```
Google(...)
```

Google is an **OAuth/OIDC provider**.

But Auth.js has many provider integrations.

For example:

```
providers: [  
  Google(...),  
  GitHub(...),  
]
```

could give you:

```
[ Continue with Google ]
```



```
[ Continue with GitHub ]
```

And the same Auth.js machinery handles the different provider flows.

---

## 26. What Auth.js are we using in Next.js?

For **current Next.js applications**, the library is:

**Auth.js's Next.js integration:** `next-auth`

The project historically was called **NextAuth.js**, and the broader project is now branded **Auth.js**.

So you'll see terminology like:

```
Auth.js  
NextAuth.js  
next-auth
```



which can be confusing.

For the current Next.js setup, think:

Auth.js



Next.js integration



npm package: next-auth



The official current docs show exactly this style:

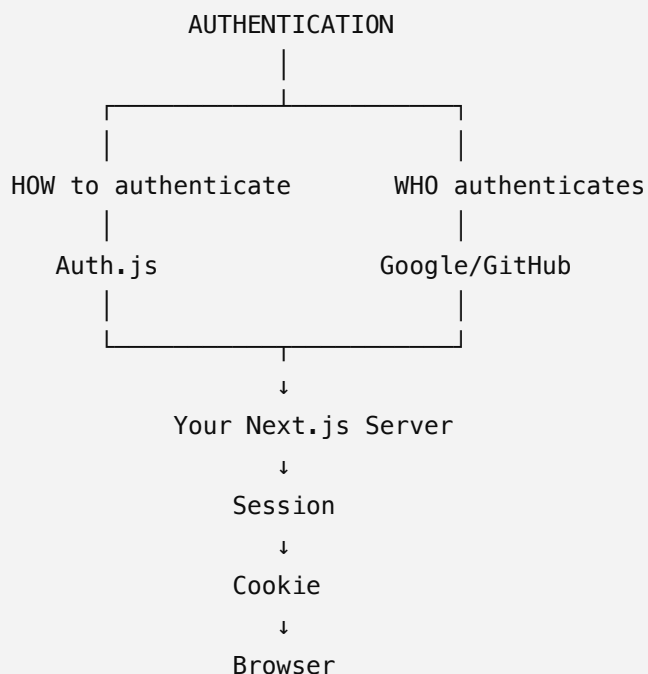
```
import NextAuth from "next-auth";
```

and:

```
import GitHub from "next-auth/providers/github";
```

with `auth.ts` exporting `auth` and `handlers` . 

## The one diagram I want you to remember



And one correction to the way we phrased it earlier:

**Auth.js doesn't "need a provider" in the sense that every authentication must use Google/GitHub.** A provider is simply one configured authentication mechanism. You can use an external OAuth/OIDC provider, credentials, or other supported mechanisms depending on the setup.

For your interview mental model, though, the **Google + Auth.js flow above** is the perfect example to understand OAuth. 🔥



meaning ? for OAuth and OpenID connect i didnt understand

You can configure Auth.js with an adapter to persist users/accounts/sessions in a database.-whats an adapter and also in whole flow where the code where we are receiving session and storing or creating??

Yes 😊 — there are **three separate things getting mixed together** here:

1. **OAuth vs OpenID Connect**
2. **What an Auth.js adapter is**
3. **Where Auth.js actually creates/stores/reads the session**

Let's rebuild the flow from scratch and make the code map to every step.

---

## 1. OAuth vs OpenID Connect — simplest possible explanation

The easiest way to remember it:

**OAuth asks:**

“Can this application get access?”

**OpenID Connect asks:**

“Who is the user?”

That's the core distinction.

---

## Imagine Google

You have your app:

MyApp



and Google.

Suppose your app says:

"I want permission to access Gopi's Google Calendar."

That's an **authorization** problem.

Google can ask Gopi:

MyApp wants access to:



- ☒ View your calendar
- ☒ Read your profile

[Allow]

That's what **OAuth** is fundamentally about:

OAuth

=

delegated authorization



---

## But what if your app says:

"I don't need your calendar. I just want to know who you are."

That's **identity**.

Google can provide information like:

</> JSON



```
{
  "sub": "123456789",
  "name": "Gopi",
  "email": "gopi@example.com"
}
```

That's where **OpenID Connect (OIDC)** comes in.

OIDC is an **identity layer built on top of OAuth 2.0**.

So:

OAuth 2.0

↓

Authorization

"Can this app access something?"

+

OpenID Connect

↓

Authentication / Identity

"Who is this user?"



## For "Login with Google"

You're generally dealing with:

OAuth 2.0 + OpenID Connect



rather than OAuth alone.

---

## 2. Now let's forget Auth.js for 2 minutes

Suppose we manually build:

Login with Google



The conceptual flow is:

Browser

↓

Your App

↓

Google

↓

User logs into Google

↓

Google

↓

authorization code

↓

Your App

↓

Google

↓



tokens / identity



Your App

The important thing is:

**OAuth gives us the authorization mechanism.**

**OIDC gives us standardized information about the authenticated user.**

That's enough for now.

### 3. Now what is an Adapter?

This one is much easier if you think about the word itself.

An **adapter** is basically:

**A piece of code that allows Auth.js to talk to your particular database.**

Imagine Auth.js says:

"I need to save this user."



But Auth.js doesn't know whether your application uses:

PostgreSQL

MySQL

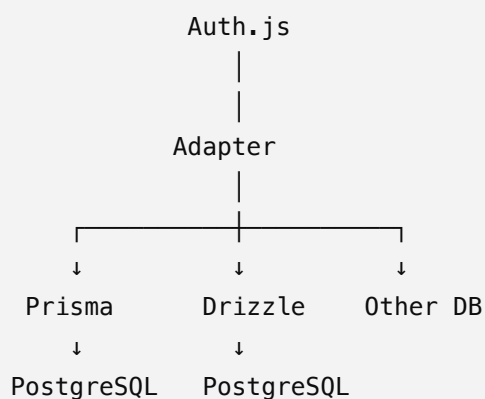
MongoDB

SQLite



or some particular ORM.

So an adapter acts as the translation layer:



So Auth.js can say conceptually:

```
adapter.createUser(...)  
adapter.getUser(...)  
adapter.createSession(...)  
adapter.getSession(...)
```



and the adapter knows how to translate that into your database operations.

## 4. Example: Prisma Adapter

Suppose your application uses:

```
PostgreSQL  
+  
Prisma
```



You could configure an Auth.js adapter.

For example, conceptually:

</> TypeScript

```
import NextAuth from "next-auth";  
import Google from "next-auth/providers/google";  
import { PrismaAdapter } from "@auth/prisma-adapter";  
import { prisma } from "@lib/prisma";  
  
export const { handlers, auth } = NextAuth({  
  adapter: PrismaAdapter(prisma),  
  
  providers: [  
    Google({  
      clientId: process.env.AUTH_GOOGLE_ID,  
      clientSecret: process.env.AUTH_GOOGLE_SECRET,  
    }),  
  ],  
});
```



The important line is:

</> TypeScript

```
adapter: PrismaAdapter(prisma)
```



You're telling Auth.js:



"When you need to persist authentication-related data, use this adapter, which knows how to talk to my Prisma database."

## 5. What does the adapter actually store?

This is where your previous question becomes important.

With a database-backed Auth.js setup, there can be several kinds of records.

Conceptually:

### User

User

-----

id  
name  
email



### Account

Account

-----

userId  
provider  
providerAccountId



This can represent:

Google account ABC

↓

Your application's user 123



### Session

Session

-----

sessionToken  
userId  
expires



So you might have:

**User**

```
id = 123
name = Gopi
email = gopi@example.com
```

**Account**

```
userId = 123
provider = google
providerAccountId = 987654
```

**Session**

```
sessionToken = XYZ123
userId = 123
expires = ...
```

The exact schema depends on the Auth.js adapter/database setup, but that's the conceptual model.

## 6. Now let's answer your BIG question

You asked:

"In the whole flow, where is the code where we receive the session and store/create it?"

Here's the important answer:

**You generally DON'T write that code yourself when using Auth.js.**

That's the point of Auth.js. 😊

You configure it:

`</>` TypeScript

```
export const { handlers, auth } = NextAuth({
  adapter: PrismaAdapter(prisma),

  providers: [
    Google(...)
  ],
});
```

And Auth.js internally handles the authentication/session machinery.

You write the **configuration**.

Auth.js contains the **implementation**.

---

## 7. Let's trace the actual flow

Let's use:

```
Next.js
+
Auth.js
+
Google
+
Prisma
+
PostgreSQL
```



Now:

---

### STEP 1 — User clicks Google login

Your Client Component:

```
</> TypeScript

"use client";

import { signIn } from "next-auth/react";

export function LoginButton() {
  return (
    <button onClick={() => signIn("google")}>
      Continue with Google
    </button>
  );
}
```



The browser initiates the Auth.js sign-in flow.

---

### STEP 2 — Auth.js redirects to Google

Auth.js knows:

```
</> TypeScript
```



```
providers: [  
  Google(...)  
]
```

So it constructs the appropriate Google authorization request.

Conceptually:

```
Browser  
  ↓  
Auth.js  
  ↓  
Google
```



## STEP 3 — User authenticates at Google

Google handles:

```
Email  
Password  
MFA  
etc.
```



Your application does **not** receive the Google password.

## STEP 4 — Google redirects back

After successful authentication:

```
Google  
  ↓  
Browser  
  ↓  
/api/auth/callback/google
```



Something conceptually like:

```
</> http  
  
GET /api/auth/callback/google?code=ABC123
```



## STEP 5 — Your Next.js Route Handler receives it

Remember our code:

```
</> TypeScript

// app/api/auth/[...nextauth]/route.ts

import { handlers } from "@auth";

export const { GET, POST } = handlers;
```

So:

```
GET /api/auth/callback/google
      ↓
    Route Handler
      ↓
    Auth.js handler
```

This is the entry point.

You didn't write:

```
</> TypeScript

if (code) {
  ...
}
```

because:

```
</> TypeScript

handlers
```

contains Auth.js's implementation.

---

## STEP 6 — Auth.js processes the callback

Auth.js internally processes the authorization code.

Conceptually:

```
authorization code
      ↓
```

```
Auth.js
  ↓
Google token endpoint
  ↓
tokens / identity information
```

Auth.js validates/processes the provider response.

Then it determines:

Who is this Google account?



## STEP 7 — Auth.js checks the database

Now the adapter becomes important.

Auth.js essentially needs to answer:

"Have I seen this Google account before?"

It can use the adapter.

Conceptually:

```
Auth.js
  |
  | adapter.getUserByAccount(...)
  ↓
Prisma Adapter
  |
  ↓
PostgreSQL
```



Maybe it finds:

Google account  
providerAccountId = 987654



associated with:

userId = 123



## 8. What if this is the user's FIRST login?

Then Auth.js may need to create the user/account records.

Conceptually:

Google identity

↓

Auth.js

↓

Adapter

↓

createUser()

↓

Database



and:

createAccount()

↓

Database



So the database could become:

User

-----

id: 123

email: gopi@example.com

name: Gopi



and:

Account

-----

userId: 123

provider: google

providerAccountId: 987654



Again:

**Auth.js calls the adapter.**

**The adapter talks to your DB.**

---

## 9. Now SESSION creation

This is the part you specifically asked about.

If you're using a **database session strategy**, Auth.js can create a session through the adapter.

Conceptually:

```
Auth.js
  ↓
adapter.createSession(...)
  ↓
Prisma Adapter
  ↓
PostgreSQL
```



Database:

```
Session
-----
sessionToken | userId | expires
XYZ123       | 123    | ...
```



So:

```
User 123
  ↑
  |
sessionToken = XYZ123
```



---

## 10. Then Auth.js sends the session cookie

Now Auth.js tells the browser to store the session identifier.

Conceptually the HTTP response contains:

```
</> http

Set-Cookie: authjs.session-token=XYZ123; HttpOnly; Secure; ...
```



The browser stores it.

So:

```
Auth.js
  ↓
```





HTTP response



Set-Cookie



Browser

This is the exact same cookie concept we learned earlier.

Auth.js isn't inventing some completely different browser mechanism.

It's still:

HTTP

+

Cookie



---

## 11. User now requests `/dashboard`

Later:

```
</> http
```

```
GET /dashboard
```

```
Cookie: authjs.session-token=XYZ123
```



Browser → Next.js.

---

## 12. Your Server Component calls `auth()`

Your code:

```
</> TypeScript
```

```
import { auth } from "@auth";
```

```
export default async function Dashboard() {  
  const session = await auth();
```

```
  if (!session?.user) {  
    redirect("/login");  
  }
```

```
  return (  
    <h1>  
      Welcome {session.user.name}
```



```
</h1>
);
}
```

This:

```
</> TypeScript
```



```
await auth()
```

is where **you ask Auth.js**:

```
"Who is the currently authenticated user?"
```

## 13. What happens internally?

Conceptually:

```
Browser
  |
  | Cookie: XYZ123
  |
  v
Next.js Server
  |
  |
  v
auth()
  |
  |
  v
Auth.js
  |
  |
  v
Read session cookie
  |
  |
  v
Find session
  |
  |
  v
Adapter
  |
  |
  v
Database
  |
  |
  v
Session XYZ123
  |
  |
  v
userId = 123
  |
```



```
↓  
User 123  
|  
↓  
session object
```

Then:

```
</> TypeScript  
  
session.user.name
```



might give:

```
Gopi
```



---

## 14. So where exactly is session creation code?

This is the crucial answer:

### Inside Auth.js.

Not here:

```
</> TypeScript  
  
// dashboard/page.tsx
```



and not here:

```
</> TypeScript  
  
// your own custom createSession()
```



You configure Auth.js:

```
</> TypeScript  
  
NextAuth({  
  adapter: PrismaAdapter(prisma),  
  providers: [...]  
})
```



and Auth.js internally performs operations through the adapter.

That's why an auth library is useful.

## 15. What YOU write vs what Auth.js writes

This distinction will make everything click.

### YOU write:

```
</> TypeScript   
  
export const {  
  handlers,  
  auth,  
  signIn,  
  signOut  
} = NextAuth({  
  adapter: PrismaAdapter(prisma),  
  
  providers: [  
    Google(...)  
  ]  
});
```

You also write your UI:

```
</> TypeScript   
  
signIn("google")
```

and your protected page:

```
</> TypeScript   
  
const session = await auth();
```

---

### Auth.js internally handles:

```
OAuth/OIDC flow   
  ↓  
callback processing  
  ↓  
provider communication  
  ↓  
user/account lookup  
  ↓  
user/account creation  
  ↓
```

```
session creation
  ↓
session lookup
  ↓
cookie handling
  ↓
session expiration
  ↓
sign-out/session invalidation
```

And the **adapter** handles the database persistence layer.

## 16. One HUGE correction to a possible mental model

Don't imagine:

```
Google
  ↓
Auth.js
  ↓
Adapter
  ↓
Database
  ↓
Auth.js
  ↓
Cookie
```



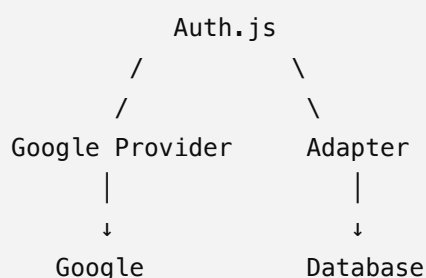
as if the adapter is another authentication service.

It's not.

The adapter is basically:

**"Auth.js, here's how you talk to my database."**

So:



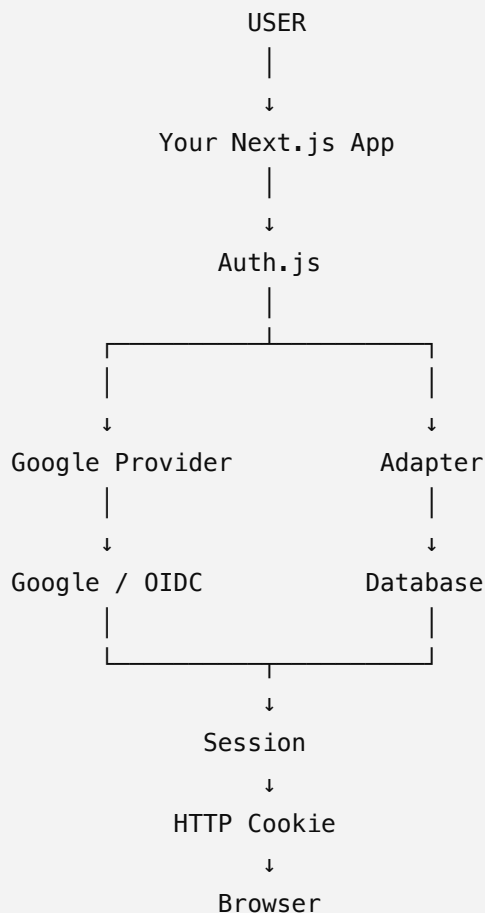
Google handles **identity authentication**.

The adapter handles **persistence**.

Auth.js coordinates the whole thing.

## 17. And now OAuth + OIDC fit in

The complete picture:



**Google/OIDC:**

"Who is this user?"

**OAuth:**

"What authorization/permission flow is being used?"

**Auth.js:**

"I'll coordinate the authentication flow and application session."

**Adapter:**

"I'll let Auth.js persist users/accounts/sessions in your database."

**Cookie:**

"I'll carry the application's session information between browser and server."

---

## And one interview answer

If they ask:

"What is an adapter in Auth.js?"

Say:

"An adapter is the persistence layer that allows Auth.js to store and retrieve authentication-related data such as users, accounts, and database sessions from my application's database. For example, with Prisma, I can configure the Prisma adapter, and Auth.js uses it internally for database operations."

And if they ask:

"Where does Auth.js create the session?"

Say:

"The application configures Auth.js with a session strategy. With database sessions, Auth.js internally uses the configured adapter to create and retrieve session records, then associates the session with the browser through a session cookie. On subsequent requests, `auth()` reads the session and resolves the current user."

That's the exact distinction I wanted you to see. 🧠🔥











































































































































































































































Awesome 🤩 Let's move to the **next important Auth.js concept**: how the session is actually used to protect pages and APIs.

## Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

```
Google
  ↓
OAuth/OIDC
  ↓
Auth.js callback
  ↓
User/Account
  ↓
Session
  ↓
Cookie
```



Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

### 1. Protecting a Server Component

Suppose we have:

```
app/
├─ login/
│   └─ page.tsx
└─ dashboard/
    └─ page.tsx
```



Our dashboard should only be accessible to logged-in users.

We can write:

```
</> TypeScript

// app/dashboard/page.tsx

import { auth } from "@auth";
import { redirect } from "next/navigation";
```



```
export default async function DashboardPage() {
  const session = await auth();

  if (!session) {
    redirect("/login");
  }

  return (
    <div>
      <h1>Welcome {session.user?.name}</h1>
      <p>{session.user?.email}</p>
    </div>
  );
}
```

Now let's understand **exactly what happens**.

## 2. User requests `/dashboard`

Browser:

```
</> http

GET /dashboard
Cookie: authjs.session-token=ABC123
```

Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript

const session = await auth();
```

runs.

## 3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript
```



```
// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```

So `auth()` is an Auth.js server-side function.

Conceptually:

```
graph TD
    Browser -- "Cookie: session-token=ABC123" --> Nextjs[Next.js]
    Nextjs --> Dashboard[Dashboard Server Component]
    Dashboard -- "auth()" --> Authjs[Auth.js]
    Authjs --> Session[Session verification / lookup]
    Session --> UserSession[User session]
```

If the session is valid:

```
</> TypeScript

session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

---

## 4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

```
</> TypeScript
```

```
const session = await auth();
```



returns something equivalent to:

```
</> TypeScript
```

```
null
```



So:

```
</> TypeScript
```

```
if (!session) {  
  redirect("/login");  
}
```



executes.

Next.js sends a redirect response.

Conceptually:

```
</> http
```

```
307 Temporary Redirect  
Location: /login
```



The browser then requests:

```
</> http
```

```
GET /login
```



So:

```
User → /dashboard
```

```
↓
```

```
auth()
```

```
↓
```

```
no session
```

```
↓
```

```
redirect("/login")
```



↓  
/login

## 5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript  
  
if (!session)
```



That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

```
</> TypeScript  
  
const session = await auth();  
  
if (!session) {  
  redirect("/login");  
}  
  
if (session.user.role !== "ADMIN") {  
  redirect("/forbidden");  
}
```



Now we're asking two different questions:

Authentication:  
"Who are you / are you logged in?"



```
Authorization:  
"Are you allowed to do this?"
```

## 6. But there's an important security rule 🚨

Never rely only on hiding UI.

For example:

```
</> TypeScript  
  
{session?.user.role === "ADMIN" && (  
  <button>Delete User</button>  
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http  
  
POST /api/users/delete
```

Therefore your server-side action/API must check authorization too.

## 7. Protecting a Route Handler

Suppose we have:

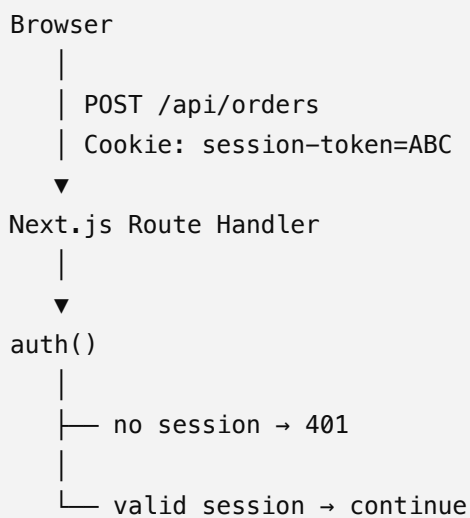
```
app/api/orders/route.ts
```

We can do:

```
</> TypeScript  
  
import { auth } from "@auth";  
import { NextResponse } from "next/server";  
  
export async function POST(request: Request) {  
  const session = await auth();
```

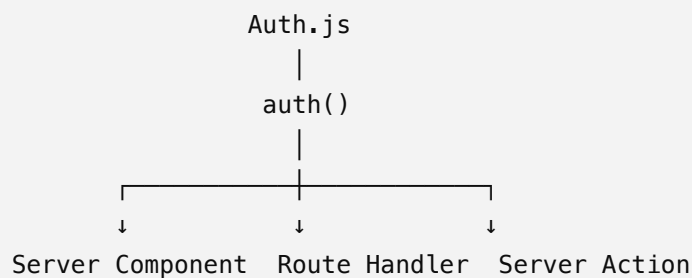
```
if (!session) {  
  return NextResponse.json(  
    { error: "Unauthorized" },  
    { status: 401 }  
  );  
}  
  
// User is authenticated  
// Continue processing order...  
  
return NextResponse.json({  
  success: true,  
});  
}
```

Now:



## 8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

### Server Component

</> TypeScript



```
const session = await auth();
```

## Route Handler

</> TypeScript



```
const session = await auth();
```

## Server Action

</> TypeScript



```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

**Your server-side code asks Auth.js to resolve the current authenticated user/session.**

## 9. Server Action example

Suppose:

</> TypeScript



```
"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

## 10. Interview answer

If an interviewer asks:

**"How would you protect a page in Next.js using Auth.js?"**

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

**"Is hiding an admin button authorization?"**

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

---

## One thing I want you to notice

We now have **three different places** where Auth.js can participate:

```
1. LOGIN
  signIn()
    ↓
  OAuth
    ↓
  callback
    ↓
  session created

2. READING SESSION
  auth()
    ↓
  "Who is currently logged in?"

3. PROTECTING ACTIONS
  auth()
    ↓
```



```
authenticate
  ↓
authorize
  ↓
perform operation
```

That distinction is **very interview-important**.

## Next after this

The natural next piece is **Auth.js callbacks** — specifically:

</> TypeScript

```
callbacks: {
  async jwt() {},
  async session() {},
  async signIn() {},
  async redirect() {}
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥

📄 🗣️ ⬆️ ↺ ...

how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

## 🔒 Remaining Auth.js / Auth & Authorization lessons

| #   | Topic               | What you'll learn                                            |
|-----|---------------------|--------------------------------------------------------------|
| 7.6 | 🛡️ Route Protection | <code>auth()</code> , protecting pages, APIs, Server Actions |



| #    | Topic                                                                                                         | What you'll learn                                                                               |
|------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 7.7  |  Auth.js Callbacks           | <code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks |
| 7.8  |  Session Customization       | Put <code>user.id</code> , <code>role</code> , etc. into the session                            |
| 7.9  |  Authorization / RBAC        | Roles, permissions, admin/user access                                                           |
| 7.10 |  Session Strategies          | Database sessions vs JWT sessions                                                               |
| 7.11 |  CSRF + SameSite             | Why state-changing requests need protection                                                     |
| 7.12 |  Middleware / Proxy Auth     | Protecting groups of routes before rendering                                                    |
| 7.13 |  Logout & Session Expiry     | Sign-out, expiration, invalid/expired sessions                                                  |
| 7.14 |  Complete Auth Architecture | Put Browser → Next.js → Auth.js → OAuth → DB → Session together                                 |
| 7.15 |  Auth Interview Questions  | Real interview scenarios + architecture questions                                               |

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 🍌 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization** section complete.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

#### AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

#### AUTHORIZATION

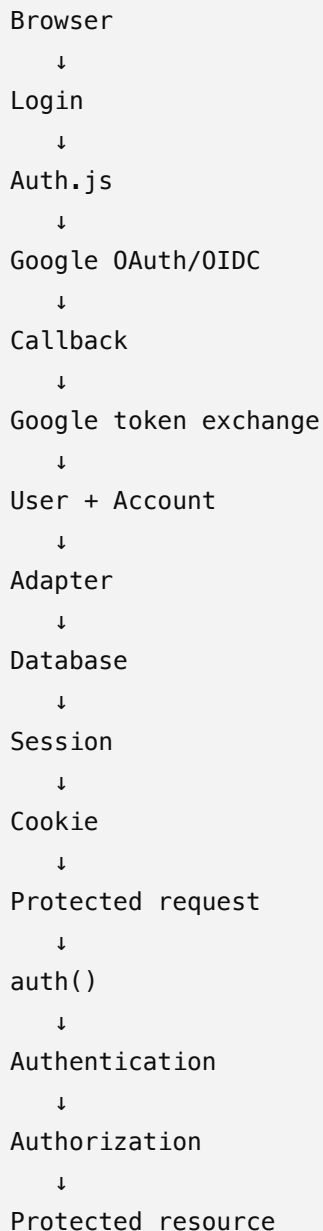
- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

## SECURITY

---

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough:**



```
graph TD; Browser --> Login; Login --> Authjs[Auth.js]; Authjs --> Google[Google OAuth/OIDC]; Google --> Callback; Callback --> GoogleToken[Google token exchange]; GoogleToken --> User[User + Account]; User --> Adapter; Adapter --> Database; Database --> Session; Session --> Cookie; Cookie --> ProtectedRequest[Protected request]; ProtectedRequest --> auth[auth()]; auth --> Authentication; Authentication --> Authorization; Authorization --> ProtectedResource[Protected resource];
```

Browser

↓

Login

↓

Auth.js

↓

Google OAuth/OIDC

↓

Callback

↓

Google token exchange

↓

User + Account

↓

Adapter

↓

Database

↓

Session

↓

Cookie

↓

Protected request

↓

auth()

↓

Authentication

↓

Authorization

↓

Protected resource

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

**So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE.** 🗝️✅



ChatGPT can make mistakes. Check important info.